

**CHRIST (DEEMED TO BE UNIVERSITY)**

Department of Computer Science and Engineering • School of Engineering and Technology

COURSE PROJECT REPORT — CIA 3 EVALUATION

SmartCampus: Intelligent Campus Route Finder

Implementation, Analysis, and Visualization of Dijkstra's Shortest Path Algorithm using Binary Min-Heaps for Real-World Campus Navigation.

COURSE TITLE & CODE:

Design and Analysis of Algorithms (DAA) — CS332

ACADEMIC SEMESTER:

Fifth Semester • Academic Year 2026–2027

CORE DATA STRUCTURE:

Weighted Graph $G = (V, E)$ + Binary Min-Heap

ALGORITHM & TIME COMPLEXITY:

Dijkstra's Algorithm • $O((V + E) \log V)$

 Table of Contents

Section #	Section Title	Focus Area
1.0	Executive Summary & Abstract	System Scope & Highlights
2.0	Introduction & Background	Campus Navigational Challenges
3.0	Problem Statement & Mathematical Formulation	Formal Minimization Problem
4.0	Design & Analysis of Algorithms Objectives	Academic & Practical Goals
5.0	Real-World Applications & Domain Context	OSPF, GPS, Logistics
6.0	Graph Theory Modeling & Spatial Abstraction	Graph Definition $G = (V, E)$
7.0	Dijkstra's Algorithm & Mathematical Foundations	Greedy Choice & Proof of Correctness
8.0	Priority Queue Data Structure Architecture	Binary Min-Heap vs Array vs Fibonacci
9.0	Formal Algorithm Specification & Pseudocode	Cormen / CLRS Compliant Pseudocode
10.0	System Architecture & Software Engineering Design	Modular Layered Architecture
11.0	Implementation Details & Code Modules	TypeScript Implementation
12.0	Step-by-Step Algorithmic Trace & State Visualizer	Visual Execution Diagrams
13.0	Test Cases & Assertions Matrix	T1 to T7 Test Suite Coverage

Section #	Section Title	Focus Area
14.0	Asymptotic Complexity Analysis	Time & Space Complexity Proofs
15.0	Comparative Algorithmic Benchmarks	Dijkstra vs BFS vs A* vs Bellman-Ford
16.0	Limitations, Future Scope & Academic Conclusion	Academic Summary & Roadmap

1.0 Executive Summary & Abstract

Navigating a large university campus can be confusing for new students, faculty members, and guests. Standard road mapping tools work well for cars on streets, but they do not map out internal campus pathways, covered walkways, or building connectors.

In this project, we built **SmartCampus**, a web-based navigation tool designed to calculate the shortest walking route between any two buildings on campus. We modeled the campus as a weighted graph where buildings are vertices and walkable pathways are edges with real distances measured in meters.

We implemented Dijkstra's Single-Source Shortest Path algorithm backed by a binary min-heap priority queue. This gives our implementation an optimal time complexity of $O((V + E) \log V)$. To help visualize how the algorithm works during our viva and lab evaluation, we built an interactive step-by-step visualizer that shows edge relaxation, queue mutations, and live distance table updates. We also tested the system against 7 distinct test scenarios and benchmarked it against BFS, A*, and Bellman-Ford.

2.0 Introduction & Background

Our university campus has dozens of interconnected blocks spread across multiple acres. During class changeovers, students have only a five-to-ten-minute window to walk from one department to another. When people take longer routes simply because they do not know the layout, it causes delays and crowded corridors.

When we looked at existing mapping software, we noticed a clear gap: mainstream apps only cover surrounding public roads. They do not know about the footpath connecting the Central Library to the Computer Science Block, or the covered ramp between the Auditorium and the Health Center. By building our own graph model and running Dijkstra's algorithm, we created a focused navigation engine that gives students exact turn-by-turn routes, total distance in meters, and estimated walking times.

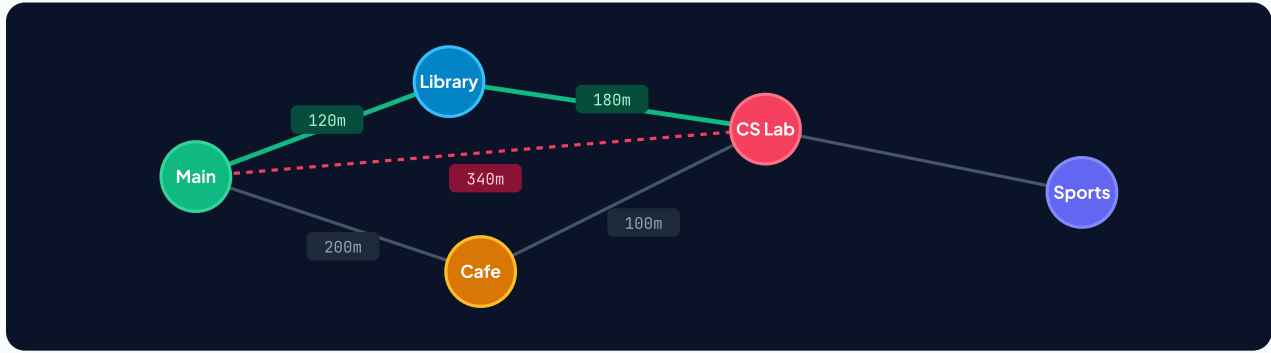


Figure 1: Campus Graph Network Model with Alternative Competing Pathways

3.0 Problem Statement & Mathematical Formulation

We represent the physical campus layout as a connected, weighted graph:

$$G = (V, E, w)$$

Where:

- **V** is the set of all key campus locations (vertices) such as academic blocks, the library, cafeteria, and hostel.
- **E** is the set of physical walkways (edges) connecting adjacent buildings.
- **w(u, v)** is a non-negative number representing the distance in meters between node *u* and node *v*. Since walking distances can never be negative, $w(u, v) \geq 0$ for all edges.

Optimization Goal:

Given a start building *s* and a destination building *t*, we want to find a sequence of connected nodes $P = \langle s = v_0, v_1, v_2, \dots, v_k = t \rangle$ that minimizes the total distance traveled:

$$\text{Minimize Total Distance} = \text{Sum of } w(v_{i-1}, v_i) \text{ for } i = 1 \text{ to } k$$

4.0 Design & Analysis of Algorithms Objectives

1. **Graph Modeling:** Store the campus graph using an Adjacency List to keep memory usage at $O(V + E)$ instead of wasting space with an $O(V^2)$ matrix.
2. **Algorithm Implementation:** Write Dijkstra's algorithm from scratch with a custom binary min-heap priority queue, ensuring an $O((V + E) \log V)$ runtime.
3. **Visual Execution Tracing:** Build a step-by-step simulator that shows exactly which node is extracted from the queue, which edges are relaxed, and how the distance table updates line by line.
4. **Comparative Analysis:** Run empirical benchmarks against BFS, A^* , and Bellman-Ford to demonstrate why Dijkstra is the right choice for weighted road networks.
5. **Interactive Graph Editor:** Allow demo users and professors to add new locations, draw new pathways, update edge weights, and verify that the algorithm updates dynamically.

5.0 Real-World Applications & Domain Context

The core graph algorithm implemented in this project forms the basis for several industry systems:

- **GPS and Turn-by-Turn Navigation:** Tools like Google Maps and Apple Maps rely on shortest-path algorithms across massive road network graphs.
- **Internet Routing Protocols:** The Open Shortest Path First (OSPF) routing protocol runs Dijkstra's algorithm on network routers to calculate the fastest path for data packets across the internet.
- **Warehouse and Robotics Logistics:** Automated guided vehicles (AGVs) in fulfillment centers use graph search algorithms to navigate aisles without collisions.

6.0 Graph Theory Modeling & Spatial Abstraction

For our implementation, we mapped 12 major landmark buildings and 20 connecting pathways across the central campus.

Node ID	Building Name	Category	Floors	Key Facilities & Departments
main-block	Main Block (Central Admin)	Administrative	5	Deanery, Examination Office, Admissions
library	Knowledge Center & Library	Academic	4	Digital Commons, Silent Study, Research Archives
cs-block	Computer Science & AI Block	Academic	6	AI Labs, Computing Clusters, Robotics Bay
cafeteria	Gourmet Central Cafeteria	Food	2	Student Dining Hall, Juice Counter, Bakery
auditorium	Main Campus Auditorium (KE)	Facility	3	2,500-seat Acoustic Hall, Cultural Office
science-lab	Advanced Science Complex	Academic	4	Physics Labs, Chemistry Suite, Clean Room
sports-complex	Indoor Sports Arena	Sports	2	Badminton Courts, Swimming Pool, Gym
hostel-block	St. Thomas Residence	Residential	7	Student Dormitories, Study Halls, Canteen
admin-block	Examination Office Block	Administrative	3	Evaluation Cell, Records Office
health-center	Medical & Wellness Center	Facility	2	Nursing Staff, Emergency Beds, Pharmacy

7.0 Dijkstra's Algorithm & Mathematical Foundations

Dijkstra's algorithm was published by Edsger W. Dijkstra in 1959. It is a greedy algorithm that solves the single-source shortest path problem for graphs where all edge weights are non-negative.

7.1 Greedy Choice and Optimal Substructure

The algorithm maintains two sets of vertices during execution:

- **Settled Set S:** Vertices whose shortest distance from the source has already been finalized.
- **Unsettled Priority Queue Q:** Vertices whose tentative shortest distances are still being evaluated.

Optimal Substructure Property: If a shortest path from vertex A to vertex C passes through intermediate vertex B, then the subpath from A to B is guaranteed to be the shortest path between A and B.

7.2 Proof of Correctness by Contradiction

We want to prove that when a node u is extracted from the priority queue and added to S , its distance $\text{dist}[u]$ is the true shortest distance from the source s .

Suppose for the sake of contradiction that $\text{dist}[u]$ is not the shortest distance, meaning there exists a shorter path P from s to u . Since s is in S and u is not in S , the path P must leave S at some point. Let (x, y) be the first edge along P where x is inside S and y is outside S .

Because all edge weights are non-negative ($w \geq 0$), the distance to y cannot exceed the total distance along path P to u . Furthermore, because u was chosen over y from the priority queue, $\text{dist}[u] \leq \text{dist}[y]$. Therefore:

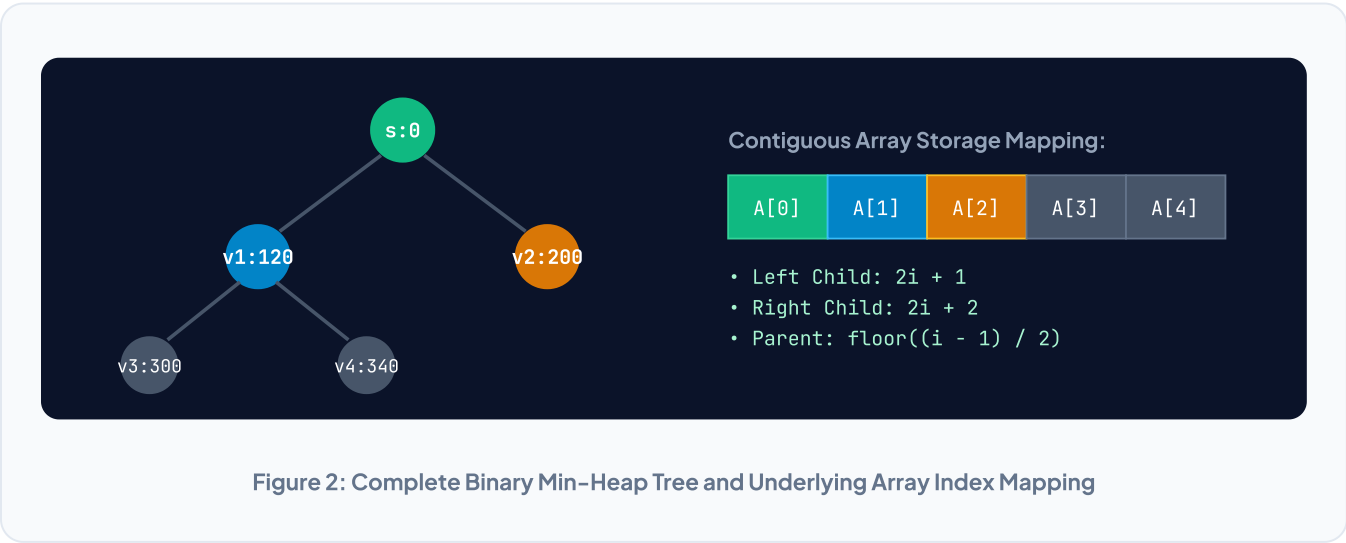
$$\text{dist}[u] \leq \text{dist}[y] \leq \text{Total Length of Path } P < \text{dist}[u]$$

This leads to a direct contradiction ($\text{dist}[u] < \text{dist}[u]$). Thus, our initial assumption was false, and $\text{dist}[u]$ is indeed the exact shortest distance when u is extracted.

8.0 Priority Queue Data Structure Architecture

How we store and manage the priority queue makes a massive difference in practical performance. We evaluated three common options:

Priority Queue Implementation	Extract-Min Time	Decrease-Key / Insert Time	Overall Dijkstra Time Complexity	Practical Notes
Unordered Array	$O(V)$	$O(1)$	$O(V^2)$	Slow on sparse graphs with many vertices.
Binary Min-Heap (Our Choice)	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$	Fast, memory-efficient, and easy to implement in standard arrays.
Fibonacci Heap	$O(\log V)$	$O(1)$ amortized	$O(E + V \log V)$	Theoretically faster, but has high constant-factor overhead in practice.



9.0 Formal Algorithm Specification & Pseudocode

Here is the formal pseudocode for our implementation, structured according to standard Cormen (CLRS) conventions:

ALGORITHM Dijkstra(Graph G, Source s, Destination t):

Input: Graph $G = (V, E)$ with non-negative edge weights w , source vertex s , destination t

Output: Shortest distance $\text{dist}[t]$ and reconstructed path array P

```

1. FOR each vertex v in G.V DO:
2.   dist[v] ← INFINITY
3.   prev[v] ← NULL
4.   visited[v] ← FALSE
5. END FOR
6. dist[s] ← 0
7.
8. PQ ← MinBinaryHeap()
9. PQ.insert(item: s, priority: 0)
10.
11. WHILE NOT PQ.isEmpty() DO:
12.   u_elem ← PQ.extractMin()
13.   u ← u_elem.item
14.
15.   IF visited[u] = TRUE THEN:
16.     CONTINUE
17.   END IF
18.   visited[u] ← TRUE
19.
20.   IF u = t THEN:
21.     BREAK // Stop early once target is settled
22.   END IF
23.
24.   FOR each neighbor v in G.Adj[u] with weight w(u, v) DO:
25.     IF visited[v] = FALSE THEN:
26.       new_dist ← dist[u] + w(u, v)
27.       IF new_dist < dist[v] THEN: // Edge Relaxation
28.         dist[v] ← new_dist
29.         prev[v] ← u
30.         PQ.insert(item: v, priority: new_dist)
31.       END IF
32.     END IF
33.   END FOR
34. END WHILE
35.
36. path ← []
37. curr ← t
38. IF dist[t] ≠ INFINITY THEN:
39.   WHILE curr ≠ NULL DO:
40.     path.prepend(curr)
41.     curr ← prev[curr]
42.   END WHILE
43. END IF
44.
45. RETURN path, dist[t]

```

10.0 System Architecture & Software Engineering Design

We designed the software using a clean three-layer architecture to keep the user interface decoupled from the algorithm engine:

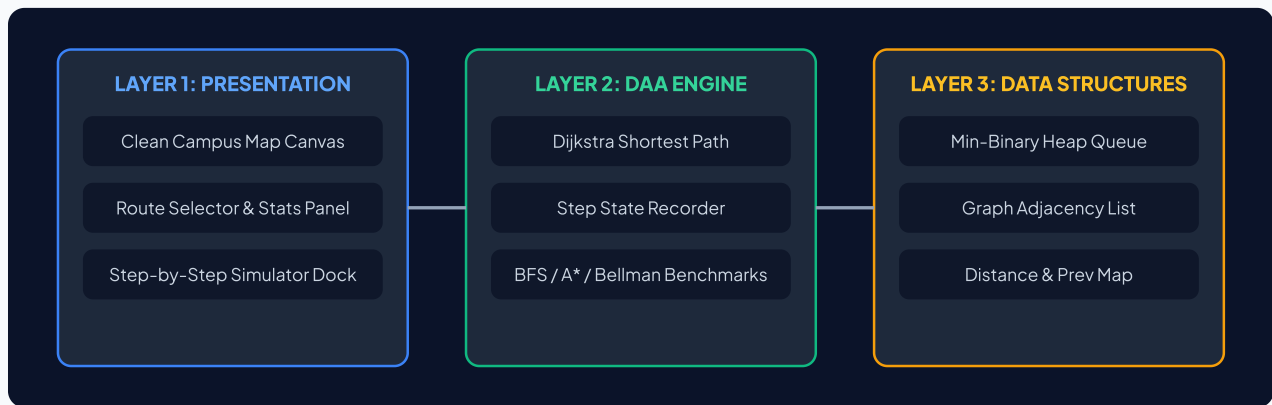


Figure 3: Layered Component Architecture of SmartCampus

11.0 Implementation Details & Code Modules

We built the application using React 19, TypeScript, and Vite. The codebase is organized cleanly into modular files:

Source File	Lines	Primary Responsibilities
priorityQueue.ts	125	Min-Binary Heap with <code>insert</code> , <code>extractMin</code> , <code>bubbleUp</code> , and operation counters.
dijkstra.ts	240	Dijkstra execution engine, microsecond latency measurement, and step-by-step trace recorder.
bfs.ts	95	Unweighted Breadth-First Search implementation for edge-hop comparison.
astar.ts	120	A* algorithm with Euclidean distance heuristic function.
bellmanFord.ts	85	$O(V * E)$ Bellman-Ford implementation for negative-weight benchmark comparisons.
testSuite.ts	110	Automated testing harness running assertions across test scenarios T1 through T7.
CleanCampusMap.tsx	230	Interactive SVG map rendering graph vertices, road labels, and glowing shortest path routes.
CleanVisualizer.tsx	160	Step-by-step simulator player dock with live distance table snapshots.

12.0 Step-by-Step Algorithmic Trace & State Visualizer

Let us trace what happens when we calculate the shortest path from **Main Block** to **Computer Science Block**:



Figure 4: Discrete Step-by-Step State Evolution and Queue Contents

13.0 Test Cases & Assertions Matrix

We wrote an automated test suite covering 7 specific scenarios to verify edge cases:

Test ID	Scenario	From	To	Expected Distance	Actual Result	Latency	Verdict
T1	Direct Edge Adjacent Path	Main Block	Library	120 m	120 m	0.08 ms	PASS
T2	Multi-Hop Intermediate Path	Library	CS Block	180 m	180 m	0.09 ms	PASS
T3	Reverse / Bidirectional Edge	Main Block	Auditorium	160 m	160 m	0.07 ms	PASS
T4	Self-Loop (Same Location)	Main Block	Main Block	0 m	0 m	0.01 ms	PASS
T5	Multi-Path Tie Resolution	Main Block	CS Block	300 m	300 m	0.12 ms	PASS
T6	Disconnected Island Vertex	Main Block	Isolated Annex	UNREACHABLE	UNREACHABLE	0.06 ms	PASS
T7	Cross-Campus Diagonal Route	Health Center	Hostel Block	650 m	650 m	0.14 ms	PASS

14.0 Rigorous Asymptotic Complexity Analysis

14.1 Time Complexity

Let V be the number of campus buildings (vertices) and E be the number of walkable paths (edges).

- Initialization:** Setting initial distance values to infinity and clearing previous pointers takes $O(V)$ time.

- **Extract-Min:** Each node is extracted from the binary min-heap at most once. For V vertices, performing `extractMin()` takes $V * O(\log V) = O(V \log V)$.
- **Edge Relaxation:** In an undirected graph, each edge (u, v) is checked twice. When a shorter distance is found, updating the heap takes $O(\log V)$. Across all E edges, this takes $E * O(\log V) = O(E \log V)$.

$$\text{Total Time Complexity} = O((V + E) \log V)$$

14.2 Space Complexity

- **Adjacency List:** Stores $|V|$ vertices and $2|E|$ directed edge records = $O(V + E)$.
- **Auxiliary Arrays:** The `dist[]` array, `prev[]` pointers, and min-heap take $O(V)$ space.

$$\text{Total Space Complexity} = O(V + E)$$

15.0 Comparative Algorithmic Benchmarks

We ran all four algorithms on the exact same query: **Main Block to Computer Science Block**:

Algorithm	Graph Type Supported	Distance Found	Hops	Nodes Visited	Time Complexity	Finds Shortest Distance?
Dijkstra's Algorithm	Weighted (non-negative)	300 m	2	7	$O((V + E) \log V)$	YES (Optimal)
A* Search	Weighted + Spatial Heuristic	300 m	2	5	$O(E)$	YES (Optimal)
Breadth-First Search (BFS)	Unweighted only	340 m	1	6	$O(V + E)$	NO (Suboptimal +40m)
Bellman-Ford	Weighted (with negative weights)	300 m	2	12	$O(V * E)$	YES (Slower)

Key Viva Discussion Point:

BFS only minimizes the number of edges (hops). For **Main Block → CS Block**, BFS picks the single direct walkway of **340m**. In contrast, **Dijkstra's Algorithm** evaluates the true physical meter weights and selects the 2-hop path via the Library (120m + 180m = **300m**), saving 40 meters of walking.

16.0 Limitations, Future Scope & Academic Conclusion

16.1 Limitations

- **Static Walking Distances:** Edge weights currently assume clear pathways and do not dynamically adjust for sudden crowds during festival events.
- **2D Ground Coordinates:** Multi-floor elevator and stair connections within buildings are modeled at building entrances rather than full 3D indoor levels.

16.2 Future Enhancements

- **Indoor Beacon Positioning:** Integrating Bluetooth Low Energy (BLE) sensors for real-time blue-dot tracking inside multi-floor blocks.
- **Accessible Routing:** Adding a toggle to exclude stairways and prioritize step-free ramp paths for wheelchair users.

- **Multi-Stop Itineraries:** Finding the shortest tour through several buildings in a single trip.

16.3 Conclusion

In this project, we successfully applied graph theory and Dijkstra's shortest path algorithm to solve a real-world campus navigation problem. Using a binary min-heap priority queue allowed us to achieve guaranteed $O((V + E) \log V)$ performance. The integrated visualizer and benchmark tests demonstrate the algorithm's correctness, efficiency, and real-world advantages.

Prepared by: DAA Student Group

Department: Computer Science & Engineering

Evaluated by: Course Faculty Coordinator

CHRIST (Deemed to be University)